

# Base de datos compartida paso a paso

Isabella Rodriguez Posada

WiseTrip

15 de septiembre, 2026



Abrimos Neon

**¿Que es Neon?**

Neon es un servicio de base de datos PostgreSQL basado en la nube y totalmente gestionado bajo un modelo serverless, diseñado para independizar el almacenamiento del procesamiento. Al hospedar la base de datos en Neon, dejamos de depender de un entorno local (localhost) y obtienes un enlace de conexión seguro y accesible desde cualquier dispositivo o servidor web con acceso a internet.

Mas que todo, para que cuando despues de una pantalla que mande informacion a la base de datos no aparezca esto:

---

## Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Wed Sep 16 00:44:43 GMT-05:00 2026

There was an unexpected error (type=Internal Server Error, status=500).

No se pudo consultar el usuario en PostgreSQL.

```
java.lang.IllegalStateException: No se pudo consultar el usuario en PostgreSQL.
    at com.wisetrip.datos.UsuarioDAO.buscarPorCorreo(UsuarioDAO.java:93)
    at com.wisetrip.servicio.UsuarioServicio.autenticar(UsuarioServicio.java:122)
    at com.wisetrip.controlador.AuthControlador.procesarLogin(AuthControlador.java:84)
    at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:104)
    at java.base/java.lang.reflect.Method.invoke(Method.java:565)
    at org.springframework.web.method.support.InvocableHandlerMethod.doInvoke(InvocableHandlerMethod.java:252)
    at org.springframework.web.method.support.InvocableHandlerMethod.invokeForRequest(InvocableHandlerMethod.java:184)
    at org.springframework.web.servlet.mvc.method.annotation.ServletInvocableHandlerMethod.invokeAndHandle(ServletInvocableHandlerMethod.java:117)
    at org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.invokeHandlerMethod(RequestMappingHandlerAdapter.java:934)
    at org.springframework.web.servlet.mvc.method.annotation.RequestMappingHandlerAdapter.handleInternal(RequestMappingHandlerAdapter.java:853)
    at org.springframework.web.servlet.mvc.method.AbstractHandlerMethodAdapter.handle(AbstractHandlerMethodAdapter.java:86)
    at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:963)
    at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:866)
    at org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1000)
    at org.springframework.web.servlet.FrameworkServlet.doPost(FrameworkServlet.java:903)
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:649)
    at org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:874)
    at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:710)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:132)
    at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:59)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:111)
    at org.springframework.web.filter.RequestContextFilter.doFilterInternal(RequestContextFilter.java:100)
    at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:111)
    at org.springframework.web.filter.FormContentFilter.doFilterInternal(FormContentFilter.java:93)
    at org.springframework.web.filter.OncePerRequestFilter.doFilter(OncePerRequestFilter.java:116)
    at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:111)
    at org.springframework.web.filter.CharacterEncodingFilter.doFilterInternal(CharacterEncodingFilter.java:199)
```

## Paso A Paso de integrar la BD local con la de la nube en Neon

Para el alcance del proyecto, donde solo usamos Postgresql, Neon actúa como una base de datos relacional estándar, solo que en una nube.

Despues de crear la cuenta, NeonBD nos da la key y el url, eso es lo que debemos poner en el codigo para que se integre.

Connect to your branch

×

Branch

production Default

Postgres database Storage Data API Auth

Compute

Primary Active

Database neondb Role neondb\_owner Reset password

Connection string Connection pooling ⓘ

postgresql://neondb\_owner:\*\*\*\*\*@ep-twilight-sky-a5jhz43b.us-east-2.aws.neon.tech/neondb?sslmode=require&channel\_binding=require

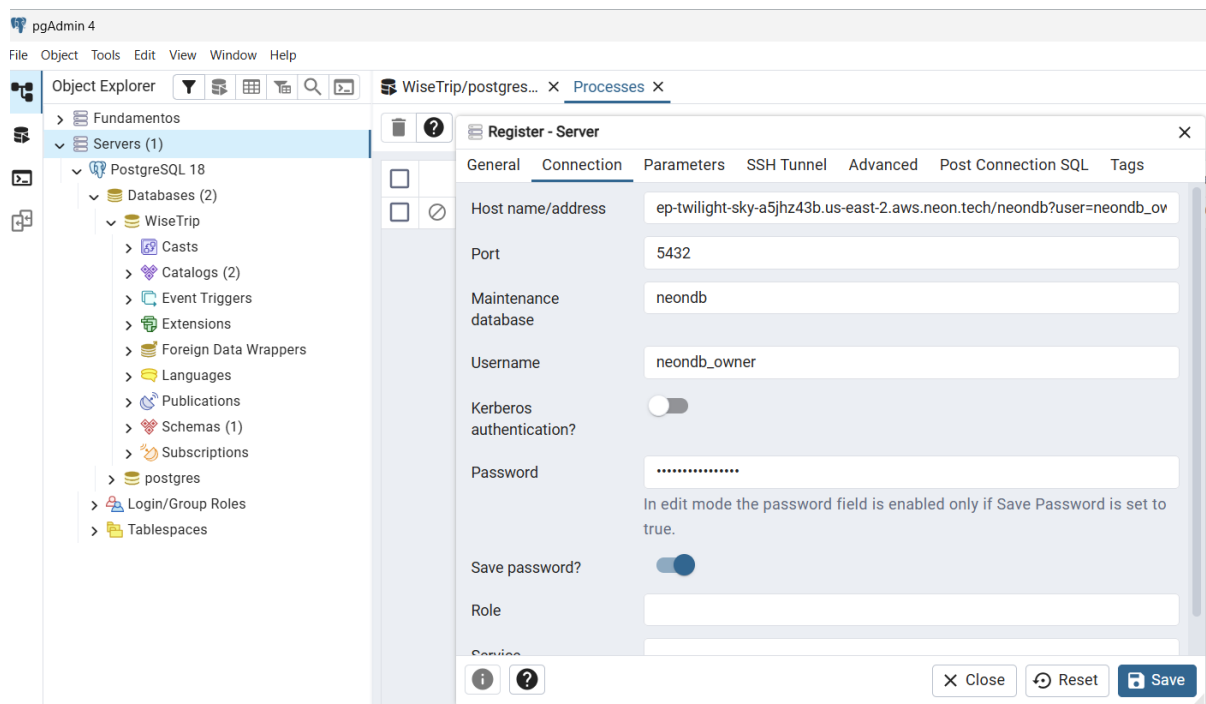
Copy snippet Show password

despues hacemos el backup en el postgres para poder exportarlo al Neon, de la base de datos local

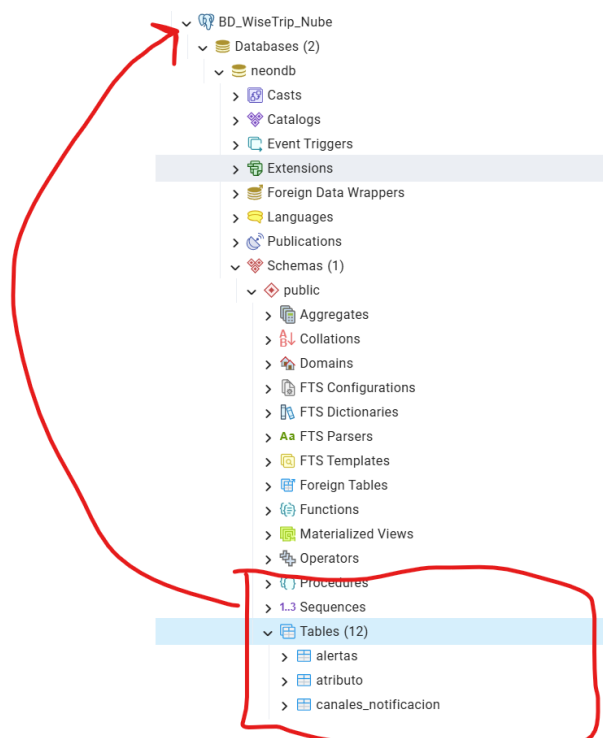
The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows a tree structure with 'Fundamentos' expanded, containing 'Servers (1)' and 'Databases (2)'. Under 'Databases (2)', 'WiseTrip' is selected. The main pane shows the 'Processes' tab for the 'WiseTrip/postgres...' server. A single process is listed with the following details:

|                          | PID   | Type          | Server                 | Object   | Start Time             | Status   | Time Taken (sec) |
|--------------------------|-------|---------------|------------------------|----------|------------------------|----------|------------------|
| <input type="checkbox"/> | 13476 | Backup Object | PostgreSQL 18 (loca... | WiseTrip | 16/9/2026, 1:01:50 ... | Finished | 0.8              |

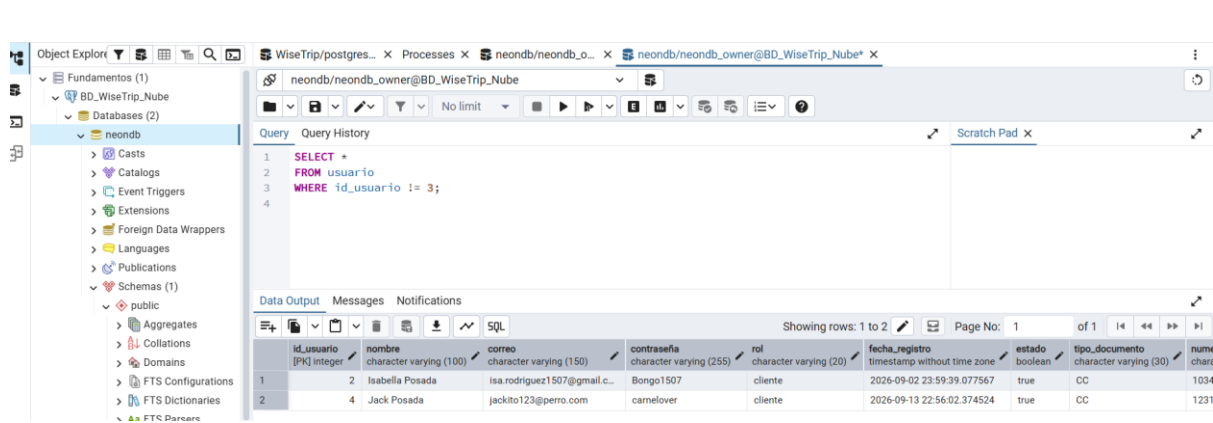
Luego creamos una nueva bd pero con lo que nos dio Neon



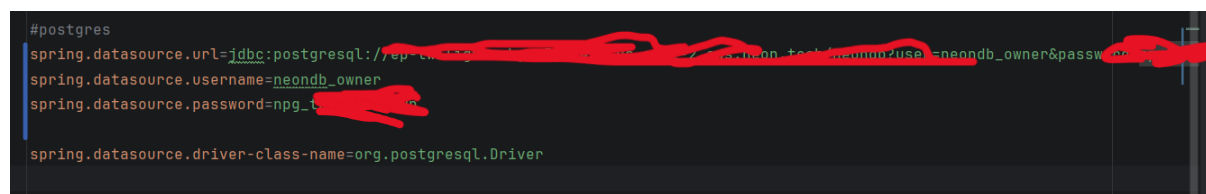
Despues ese Backup que hicimos del local lo ponemos en la que hicimos con el Neon, cuando termine, se pueden ver en el database de neon las tablas que habia hecho desde local



Acá probamos con un query en la base de datos del Neon y nos responde perfectamente



Y ya ahora cambiamos en el código el application.properties con la contraseña y el url que nos brindo el NeonBD



Por temas de seguridad lo censuro, pero realmente está en el código

Para las pruebas, tengo que pedirle a alguno de mis compañeros que abra el localhost desde otro dispositivo, pero quedara pendiente.

### Gestión de la base de datos compartida:

Una vez que WiseTrip utilice Neon como base de datos compartida, esta será la fuente principal de información para todo el equipo. Los cambios realizados sobre la estructura o los datos podrán hacerse desde pgAdmin conectado a Neon, desde el editor SQL de Neon o directamente desde la aplicación cuando esta registre o actualice información. De esta manera, todos los integrantes trabajarán sobre la misma base de datos y podrán visualizar los cambios realizados por los demás.

Para modificaciones importantes en la estructura de la base de datos, como creación de tablas, nuevas columnas o relaciones, se recomienda conservar también los scripts SQL dentro del repositorio del proyecto. Esto permitirá mantener un historial de los cambios realizados, facilitar su revisión y reconstruir la base de datos si fuera necesario.

La base PostgreSQL local podrá mantenerse únicamente como respaldo o entorno personal de pruebas, mientras que Neon será la base de datos oficial del proyecto.

### Acceso y edición para mis compañeros

Para que mis compañeros puedan acceder a la base de datos compartida deben hacerlo desde el neonBD. La base de datos oficial de WiseTrip se encuentra alojada en Neon, por lo que todos los integrantes del equipo pueden trabajar sobre la misma información de forma remota.

En mi caso, como ya configuré la conexión de Neon en pgAdmin, puedo acceder directamente desde PostgreSQL a neonBD, visualizar sus esquemas y tablas, ejecutar queries y realizar modificaciones sobre la estructura o los datos.

Para los demás integrantes del equipo existen dos formas de acceder y editar la base de datos:

1. **Desde Neon:** cada integrante debe ser invitado al proyecto WiseTripBD con permisos de edición. Una vez dentro del proyecto, podrán utilizar las secciones Tables y SQL Editor para consultar datos, modificar registros, crear nuevas tablas, agregar columnas o realizar otros cambios sobre la base de datos compartida.
2. **Desde pgAdmin:** cada integrante puede registrar una conexión utilizando el host, puerto, nombre de la base de datos, usuario, contraseña y configuración SSL proporcionados por Neon. Una vez conectados, podrán visualizar las mismas tablas y esquemas disponibles en neondb y realizar cambios directamente desde su herramienta local.

Cualquier modificación realizada sobre la conexión de Neon se verá reflejada para todos los integrantes del equipo, ya que todos trabajan sobre la misma base de datos compartida. Por esta razón, los cambios importantes de estructura deben documentarse también mediante scripts SQL dentro del repositorio de GitHub, con el fin de mantener un historial y facilitar la recuperación o reconstrucción de la base si fuera necesario.